

RAG

Introduction

대규모 사전학습 Language Model은 많은 factual knowledge를 parameter 내부에 저장하고 있음.

하지만 모든 지식을 parameter 안에만 저장하는 방식에는 문제가 존재.

- 새로운 지식이 생겼을 때 model weight를 다시 학습해야 함.
- 어떤 knowledge를 근거로 답했는지 확인하기 어려움.
- knowledge-intensive task에서는 parameter 내부 지식만으로 부족할 수 있음.
- 잘못된 사실을 그럴듯하게 생성하는 hallucination 문제가 존재.

즉, **Model 내부의 Parametric Memory + 외부에서 검색하는 Non-Parametric Memory**를 같이 사용해보자가 RAG의 출발점.

Main Goal

Pre-trained seq2seq model이 가지고 있는 parametric knowledge에 외부 문서 검색 결과를 추가하여,

검색(Retrieval) + 생성(Generation)을 하나의 모델 안에서 결합하는 것.

기존 LM:

Input → Language Model → Output

RAG:

Input → Retriever → 관련 문서 검색 → Generator → Output

Motivation

Parametric Memory

Language Model의 weight 안에 학습되어 있는 지식.

예를 들어 BART가 pre-training 과정에서 학습한 언어적/사실적 지식 등이 여기에 해당.

장점은 별도의 검색 없이 바로 생성할 수 있다는 것.

하지만 지식을 수정하거나 추가하려면 model parameter 자체를 다시 학습해야 하는 문제가 있음.

Non-Parametric Memory

외부에 저장되어 있는 문서 collection을 직접 검색해서 사용하는 방식.

이 논문에서는 **Wikipedia passage**를 **dense vector**로 만들어 **index**에 저장함.

Model이 모르는 정보가 있더라도 외부 문서를 검색해서 가져올 수 있음.

또한 문서 index를 바꾸는 것만으로 knowledge를 업데이트할 수 있음.

즉,

Parametric Memory = Model Weight

Non-Parametric Memory = External Document Index

RAG는 이 둘을 같이 사용하는 방식.

Method

RAG는 크게 두 개의 component로 구성됨.

1. **Retriever**
2. **Generator**

입력 query를 x , 검색된 문서를 z , 최종 생성 결과를 y 라고 하자.

Retriever:

$$p_{\eta}(z|x)$$

Generator:

$$p_{\theta}(y_i|x, z, y_{1:i-1})$$

- x : 입력 query
- z : Retriever가 검색한 passage
- y_i : 현재 생성하려는 token
- $y_{1:i-1}$: 이전까지 생성된 token
- η : Retriever의 parameter
- θ : Generator의 parameter

즉 Retriever가 먼저 x 와 관련 있는 문서 z 를 찾고,

Generator는

$$x + z$$

를 같이 보고 답변 y 를 생성함.

Retriever: DPR

Retriever는 **Dense Passage Retriever(DPR)**를 사용.

Query와 Document를 각각 BERT Encoder로 vector화함.

$$\mathbf{q}(x) = BERT_q(x)$$

$$\mathbf{d}(z) = BERT_d(z)$$

그리고 Query vector와 Document vector의 inner product가 클수록 관련 있는 문서라고 판단.

$$p_{\eta}(z|x) \propto \exp(\mathbf{d}(z)^T \mathbf{q}(x))$$

쉽게 말하면,

Query → Vector

Document → Vector

로 만든 뒤 vector가 가까운 문서를 검색함.

전체 Wikipedia를 매번 다 비교할 수는 없으므로 **MIPS(Maximum Inner Product Search)**를 사용해 top- K passage를 가져옴.

논문에서는 document encoder와 index는 고정하고, query encoder를 fine-tuning함.

Generator: BART

Generator는 **BART-large**를 사용.

BART는 encoder-decoder 기반의 pre-trained seq2seq Transformer.

Retriever가 가져온 document z 와 원래 input x 를 concatenate해서 BART에 입력함.

대략,

```
Question x
↓
Retriever
↓
```

```

Top-K Documents z
↓
[x ; z]
↓
BART Generator
↓
Answer y

```

형태.

RAG-Sequence vs RAG-Token

논문에서는 두 가지 RAG formulation을 제안함.

RAG-Sequence

하나의 output sequence 전체를 생성할 때 같은 document를 사용한다고 가정.

$$p_{\text{RAG-Sequence}}(y|x) \approx \sum_{z \in \text{Top-K}} p_{\eta}(z|x) p_{\theta}(y|x, z)$$

즉 여러 문서를 가져오긴 하지만,

각 document가 전체 답변을 얼마나 잘 설명하는지를 계산해서 합치는 방식.

예를 들어 답변이 10 token이라면 하나의 document z 를 기준으로 10 token 전체를 생성한다고 보는 것.

RAG-Token

RAG-Token은 각 token마다 다른 document를 참고할 수 있음.

$$p_{\text{RAG-Token}}(y|x) \approx \prod_i \sum_{z \in \text{Top-K}} p_{\eta}(z|x) p_{\theta}(y_i|x, z, y_{1:i-1})$$

즉 첫 번째 token은 문서 A를 많이 참고하고,

다음 token은 문서 B를 많이 참고하는 식으로 여러 document의 information을 token level에서 섞을 수 있음.

핵심 차이:

- **RAG-Sequence:** 하나의 sequence 단위로 document 선택
- **RAG-Token:** token마다 서로 다른 document 사용 가능

Training

검색되는 document z 는 정답으로 직접 주어지는 것이 아니라 **latent variable**로 취급함.
즉 학습 데이터는 기본적으로

$$(x, y)$$

input-output pair만 있고,

“이 query에서는 정확히 이 document를 검색해야 한다”라는 document supervision이 반드시 필요한 것은 아님.

Retriever와 Generator를 함께 학습하면서 최종 정답 y 를 잘 생성하도록 최적화함.

Objective는 negative marginal log-likelihood를 최소화.

- $-\log p(y|x)$

논문에서는

- Document Encoder: Fixed
- Wikipedia Index: Fixed
- Query Encoder: Trainable
- BART Generator: Trainable

형태로 학습.

Document encoder까지 계속 바꾸면 모든 document embedding을 다시 계산해서 index를 업데이트해야 하므로 비용이 큼.

Experiments

Knowledge-intensive NLP task에서 RAG를 평가함.

대표적으로:

- Natural Questions
- TriviaQA
- WebQuestions
- CuratedTREC
- MS-MARCO

- Jeopardy Question Generation
- FEVER

Open-Domain QA

RAG는 Natural Questions, WebQuestions, CuratedTREC 등의 open-domain QA에서 당시 강한 성능을 보임.

중요한 점은 기존 retrieve-and-extract 방식처럼 문서에서 answer span만 뽑는 것이 아니라,

검색된 document를 기반으로 자유롭게 answer를 generation한다는 것.

Generation Quality

Parametric-only BART와 비교했을 때 RAG가 더

- factual
- specific
- diverse

한 output을 생성하는 경향을 보임.

즉 외부 knowledge를 직접 참고하기 때문에 model parameter 안의 기억에만 의존하는 generation보다 knowledge-intensive generation에 유리함.

Knowledge Update

RAG의 중요한 장점 중 하나.

Model weight를 다시 학습하지 않고 **Document Index 자체를 교체**할 수 있음.

즉 새로운 Wikipedia snapshot으로 index를 바꾸면 model parameter를 수정하지 않아도 새로운 knowledge를 retrieval할 수 있음.

핵심 정리

RAG의 핵심은 단순히 “검색하고 LLM에게 넣는다”가 아님.

논문에서는 Retriever와 Generator를 하나의 probabilistic framework 안에서 결합하고,

검색된 document를 latent variable로 두어 end-to-end fine-tuning함.

전체 흐름:

```
Input x
  ↓
Query Encoder
  ↓
Dense Retrieval
  ↓
Top-K Documents z
  ↓
Input x + Document z
  ↓
BART Generator
  ↓
Output y
```

결국,

Language Model의 Parametric Memory만 사용하는 것이 아니라 External Knowledge를 Non-Parametric Memory로 추가해 generation에 활용하는 방법.

RAG 이후 LLM system에서 사용하는

Query → Retrieval → Context → Generation

구조의 기본 아이디어가 됨.

Paper

- Patrick Lewis et al., *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*, NeurIPS 2020
- <https://arxiv.org/abs/2005.11401>